

Full Length Article

FuzzAGG: A fuzzing-driven attack graph generation framework for industrial robot systems

Xiaosheng Liu, Wenqi Jiang, Zhongwei Li^{*}, Xianji Jin, Zihan Ma, Qingyang Li

School of Electrical Engineering and Automation, Harbin Institute of Technology, Harbin, PR China

ARTICLE INFO

Keywords:

Industrial robot systems
Fuzzing
Risk data modeling
Domain specific language
Attack Graph Generation

ABSTRACT

As industrial robot systems (IRS) are increasingly utilized in smart factories, their information security issues have become particularly critical. Attack graphs, an essential system-level risk modeling technique, traditionally rely on predefined risk attributes and exploitation rules for their generation. However, this approach fails to meet the needs for attack graph generation and analysis in environments with missing risk data. To address this issue, this paper proposes a fuzzing-driven attack graph generation framework, FuzzAGG. This framework aims to provide an efficient and accurate method for generating attack graphs under conditions of incomplete risk data, thereby supporting information security analysis and risk assessment of IRS. In this paper, a risk data model (RDM) is constructed using the Meta Attack Language to achieve a structured description of the risk data of IRS. A fuzzing test case generation algorithm based on the MU-SeqGAN model is proposed, which can generate test cases suitable for the state machines of IRS and map them to specific Risk Data Model Objects (RDMOs). Additionally, a conversion unit is designed to integrate all RDMOs into a risk description file, which is then used by the generation unit to construct a graphical attack graph. In performance tests, FuzzAGG is able to achieve automated construction of IRS attack graphs containing 1000 state nodes in 42 min and maintain 88 % risk coverage. Taking the IRS of a PCB automated production line the effectiveness of the FuzzAGG framework is validated. The results demonstrate that FuzzAGG can automatically generate and validate an attack graph containing 184 attribute nodes and atomic attack nodes in 8 min with high operational efficiency, proving the practicality and reliability of this method in automated attack graph generation.

1. Introduction

With the development of industrial robot technology, the use of crucial industrial automation equipment such as robotic arms and AGVs in smart factories has become increasingly common. Industrial robots not only enhance production efficiency and promote automation and precision within the industrial sector but also reduce accidents caused by human error. However, as IRS connect to the internet and extensively utilize traditional information and communication technologies, they face increasing cybersecurity challenges. These systems may be subjected to highly targeted and persistent cyberattacks, leading to severe consequences such as production interruptions, equipment damage, or loss of critical data. Traditional cybersecurity measures are inadequate in addressing the risks and attack behaviors specific to IRS, necessitating customized cybersecurity solutions. IRS are characterized by a variety of device types and compact network structures, making the intuitive display and analysis of potential risks and attack paths an essential

prerequisite for effective security protection. Attack graph technology identifies potential attacks within the system, providing structured modeling and visual descriptions of the interrelationships between devices and risks. This helps security analysts understand risk exploitation methods and assess their impacts, thereby offering an effective tool for the security analysis of IRS. Despite the advantages of attack graphs in illustrating risk distribution and attack patterns, existing generation methods rely on predefined risk exploitation rules. These methods require significant human involvement in risk data processing and model construction, and the complexity of device types, network scale, topology, business processes, and functional configurations within the system poses challenges to the computational efficiency and scalability of automatic attack graph generation. Furthermore, compared to information systems, the risk data within IRS requires more specialized knowledge in industrial control system security for collection and integration. The inability of various manufacturers of industrial robots and auxiliary equipment to provide complete, structured device data further

^{*} Corresponding author.

E-mail address: lzw@hit.edu.cn (Z. Li).

<https://doi.org/10.1016/j.cose.2024.104223>

Received 4 July 2024; Received in revised form 14 October 2024; Accepted 19 November 2024

Available online 20 November 2024

0167-4048/© 2024 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

complicates the attack graph generation process, which must be conducted under constraints of incomplete risk data, potentially leading to incomplete coverage and inaccurate node link relationships in the attack graphs.

To address these issues, this paper presents FuzzAGG, a fuzz testing-driven framework designed to generate attack graphs for IRS, addressing challenges associated with incomplete risk data. The core contribution includes the development of a comprehensive risk data model that systematically describes the relationships between attack behaviors and risk attributes in IRS. By integrating MU-SeqGAN-based fuzz testing, the framework ensures high-coverage testing of IRS state machines, enabling the identification of potential vulnerabilities and attack paths. A case study on a PCB automated production line validates the framework's effectiveness, where FuzzAGG successfully generated an attack graph with 184 nodes, demonstrating its operational efficiency and precision. This framework offers a significant advancement in IRS security analysis, automating the generation of attack graphs, improving vulnerability detection, and enabling better risk assessment, even in environments where risk data is incomplete or difficult to obtain. The remainder of this paper is organized as follows: Section 2 discusses related work; Section 3 introduces the composition of IRS and the theory of risk data modeling; Section 4 proposes the fuzz testing-driven attack graph generation framework; Section 5 tests and validates the proposed method using the industrial robotic system of a PCB automated production line as an example; and the final section summarizes the research work and provides security protection recommendations for IRS. The main contributions of this paper include:

- (1) Constructing a risk data model for IRS to structurally describe the risk attributes of IRS and the logical relationships between attack behaviors and risk exploitation processes.
- (2) Designing a fuzz testing-driven attack graph generation framework for IRS, detailing its components and their functions.
- (3) Proposing a fuzz testing case generation algorithm based on MU-SeqGAN for industrial robotic system state machines, addressing the high-coverage completion of missing risk data in IRS and enabling the mapping and conversion of fuzz testing cases into risk data model objects.

2. Related work

2.1. Attack graph framework

Attack graph technology, originating from probabilistic risk assessment techniques such as fault trees and event trees, quantifies the impact of specific components on the overall security of a system. However, these techniques have significant limitations in global analysis and integration of system vulnerability information. The concept of attack graphs was first introduced in the 1990s, combining attack sets with physical network topology to perform network vulnerability analysis (Phillips and Swiler, 1998). As attack graph technology has evolved, the definitions of nodes and directed edges within attack graphs have been continuously refined, leading to the development of various attack graph frameworks.

State attack graphs, one of the early frameworks, were proposed by (Homer et al., 2013). In these graphs, nodes represent the state information of system assets, and directed edges describe state transitions. However, as the scale of network assets grows and the complexity of asset states increases, the number of state nodes in state attack graphs increases exponentially, resulting in a significant rise in analysis computation time. Additionally, considering the dynamic nature of the network environment in IRS, the increase in state nodes poses substantial challenges for dynamic updates and real-time analysis of state attack graphs. To address the state explosion problem, researchers proposed attribute-based attack graph modeling methods, known as attribute attack graphs (Nia et al., 2019). This method replaces state

nodes of state attack graphs with system resources and atomic attacks by assuming the monotonicity of the attacker's capabilities. Attribute attack graphs are fast to generate, simple in structure, and highly scalable. However, they face issues such as cyclic attack paths, which cannot be completely eliminated by simply removing atomic attack nodes. Furthermore, the implicit description of attack paths in attribute attack graphs poses challenges for network security analysis.

Given that attribute attack graphs contain multiple node types, some scholars have proposed reducing the number of node types to improve their intuitive descriptive capability. For instance, (Li et al., 2006) suggested removing resource nodes to convert attribute attack graphs into penetration dependency attack graphs, which clearly describe the dependency relationships of atomic attacks but cannot fully describe the logical relationships between resources. (Ammann et al., 2002) transformed atomic attacks into directed edges connecting attributes, converting attribute attack graphs into attribute dependency attack graphs that clearly display the causal relationships between attributes during attacks, but cannot explicitly describe the attack process. The distinction between full attack graphs and partial attack graphs is crucial in security analysis. Full attack graphs aim to evaluate all potential interrelationships among known vulnerabilities within a network, providing a comprehensive view that facilitates the assessment of the overall security posture. For example, (Chen et al., 2009) describes methods to overcome the challenges of modeling and scalability in large networks, emphasizing that full attack graphs are more suitable for comprehensive vulnerability analysis in complex environments. Conversely, partial attack graphs focus on specific attack objectives, depicting only the relevant vulnerabilities and attack paths necessary to achieve particular targets. This approach reduces complexity and is frequently employed in practical scenarios where targeted defense measures are required. (Haque et al., 2017) proposes evolutionary approaches to optimize attack modeling by balancing the detailed analysis provided by full attack graphs with the efficiency of partial attack graphs. Additionally, (Ingols et al., 2006) supports the use of partial attack graphs for rapidly identifying and mitigating critical vulnerabilities in large-scale networks. (Al-Eiaddeh and Abdallah, 2024) introduces the GeniGraph model, implementing adaptive and scalable security defense strategies that integrate multiple defense scenarios derived from key attack paths identified in both full and partial attack graph contexts.

State attack graphs and attribute attack graphs can only perform qualitative analysis of system risks. To compare the impact of attack paths within attack graphs on the system, (Liu and Man, 2005) integrated Bayesian networks with attribute attack graphs. This integration endows attribute attack graphs with probabilistic reasoning capabilities, enabling the computation of conditional probabilities for all nodes in the attack graph based on the initial probabilities of each node and the causal relationships between nodes. This hybrid is termed a Bayesian attack graph.

Attack graph frameworks exhibit stronger extensibility by addressing the security characteristics of different modeling objects. For instance, to solve the problem of information security modeling in OT environments, (Barrère et al., 2023) proposed a cyber-physical attack graph, which logically encodes the exploitation of both network and physical vulnerabilities by attackers. (Bouom et al., 2023) addressed network resilience assessment issues using a logical attack graph framework, helping security analysts identify critical vulnerabilities that hinder system recovery and formulating protective decisions. Addressing the limitation of previous attack graphs that did not consider zero-day vulnerabilities, (Zhang et al., 2016) proposed a zero-day attack graph framework, which evaluates network security by assuming the existence of zero-day vulnerabilities in remote services. To incorporate social engineering vulnerabilities, (Beckers et al., 2014) proposed a social engineering attack graph that can analyze patterns of social engineering attacks with unforeseen characteristics.

In summary, current attack graphs must overcome challenges such as state explosion, cyclic attack paths, implicit description methods, and

the accuracy of probabilistic reasoning. Enhancing their extensibility based on specific system security analysis requirements is crucial to address the diverse challenges in network security.

2.2. Attack graph generation

Currently, security analysts commonly build graphical security analysis models, such as attack trees and attack graphs, manually. However, in the context of large-scale complex system security analysis, this approach often leads to issues like high computational demand, incomplete models, and incorrect node link predictions. Therefore, efficient and accurate automatic generation of attack graphs has become a popular research topic in academia. Attack graph generation strategies can be categorized into model-driven, analysis-driven, and vulnerability-driven approaches (Konsta et al., 2024). These methods require system information as a basic input and derive system attack models by integrating additional information, such as system vulnerabilities and security analysis results, according to different strategies. Some studies adopt multiple generation strategies simultaneously to improve efficiency and the completeness of attack graphs.

Model-driven strategies use formal system descriptions and predefined transformation rules to form attack models, identifying potential security threats and vulnerabilities in the system. (Ivanova et al., 2016) proposed converting graphical system models into graphical attack models, providing transformation rules between the two models to describe potential attack behaviors in the system. Another study leveraged the advantages of graphical attack models and symbolic features to automate the conversion of attack trees into attack graphs, enhancing the generality of graphical attack models (Schiele and Gadyatskaya, 2021). Although model-driven strategies generate highly accurate attack graphs due to their reliance on the accuracy of the system model itself, these methods are highly dependent on high-quality input models and struggle to adapt to dynamically changing system environments.

Analysis-driven strategies conduct in-depth analyses of system security properties through formal model verification, machine learning-based planning algorithms, and other methods to discover exploitable risks and attack paths. One example is the A2G2 V component, which automatically generates and visualizes attack graphs using model checking (Al Ghazo et al., 2019). It includes an architectural description tool that extracts formal descriptions of atomic attacks, preconditions, and postconditions and uses a model checker to automatically identify attack sequences. (Bezawada et al., 2019) proposed an attack graph generator based on intelligent planning using vulnerability text descriptions, capable of generating more detailed incremental attack graphs as the system changes. While analysis-driven strategies can discover new attack paths and adapt to risk evolution, their complex and intensive analysis processes result in relatively low attack graph generation efficiency, requiring substantial expertise and experience.

Vulnerability-driven strategies construct attack graphs using known vulnerability information and system vulnerability databases. (Tippenhauer et al., 2014) integrated various security information into a graphical representation, using logical relationships from input information to infer attack graph topology, suitable for workflow-type systems. (Koo et al., 2022) incorporated machine learning and deep learning into the attack graph generation process, using system topology information as input data and training machine learning models with vulnerability databases. While vulnerability-driven strategies can quickly identify risk exploitation patterns from known vulnerabilities, they cannot cover all possible attack scenarios comprehensively. Additionally, this strategy relies on the quality and completeness of the vulnerability database, requiring extra manual validation to ensure the generated results align with real-world scenarios and security requirements.

Based on a comprehensive analysis of the limitations in existing research, this paper proposes a fuzz testing-driven attack graph

generation framework that primarily supports the generation of attribute-based attack graphs. This type of attack graph uses attribute nodes and atomic attack nodes to intuitively illustrate how attackers can exploit system resources and execute attack operations, making it particularly suitable for addressing the complex risk data challenges in IRS. The proposed framework enables the generation of high-coverage attack graphs even in the presence of incomplete risk data. The effectiveness of the framework is validated through a case study involving a PCB automated production line. The FuzzAGG framework's components directly related to generating attack graphs for IRS include the fuzz testing unit, conversion unit, and attack graph generation unit. In the fuzz testing unit, an extended FSM model is used, along with the proposed MU-SeqGAN based method, to construct test cases suitable for the IRS's FSM. These test cases can trigger risk states in the IRS and extract key parameters for the Risk Data Model Object (RDMO) instances. The conversion unit processes the RDMO to ensure consistency and completeness of the risk information, mapping critical attributes such as assets, threats, and vulnerabilities to a standardized format. This forms a structured risk description that details IRS-related risk attributes, attack paths, and resource utilization information. Finally, the attack graph generation unit constructs a graphical representation of the IRS attack graph. In addition, the FuzzAGG framework provides a syntax checker and a model checker to ensure that the generated RDMOs comply with predefined syntax rules and that the attack graph meets standards of node reachability, path completeness, and absence of logical errors. Through the integration of these components, the FuzzAGG framework ensures a high level of accuracy and reliability in the generated IRS attack graphs.

3. IRS analysis and risk modeling

To construct an attack graph for IRS, this section provides an in-depth analysis of the IRS components and their potential risks, extracting risk characteristics unique to IRS. These characteristics differ significantly from those found in traditional information systems and are essential for accurately building a structured Risk Data Model (RDM). The RDM is a core component of the FuzzAGG framework, and its effectiveness directly determines the precision and efficiency of the subsequent attack graph generation. This section introduces a structured risk data modeling approach based on the Meta Attack Language (MAL). Unlike existing methods, this approach demonstrates considerable flexibility and innovation in handling incomplete or missing risk data, overcoming challenges faced in the automatic generation of attack graphs. Through this modeling technique, IRS risk attributes are systematically described, enabling seamless integration with subsequent fuzz testing and attack graph generation techniques. Additionally, this section includes customized modeling for the unique threats and attack vectors associated with IRS. This process ensures that the generated attack graph accurately reflects the security requirements of IRS, thereby significantly enhancing the FuzzAGG framework's effectiveness and applicability in real-world scenarios.

3.1. Industrial robot system components and risk analysis

According to the ISO 8373 definition, an industrial robot is an electromechanical system composed of a multi-axis manipulator, a control system, a human-machine interface (HMI), and hardware and software communication interfaces. It is divided into mechanical and control parts. An industrial robot system, with the industrial robot as the main component, is a modern industrial control system (ICS). Its typical network structure is shown in Fig. 1. Referring to the ISA-95 standard, the OT system of an industrial robot system can be divided into three layers: the field device layer, the field control layer, and the process monitoring layer, from bottom to top. The industrial robot controller performs core mechanical control functions, including tasks such as manipulator path planning and execution of manufacturing programs.

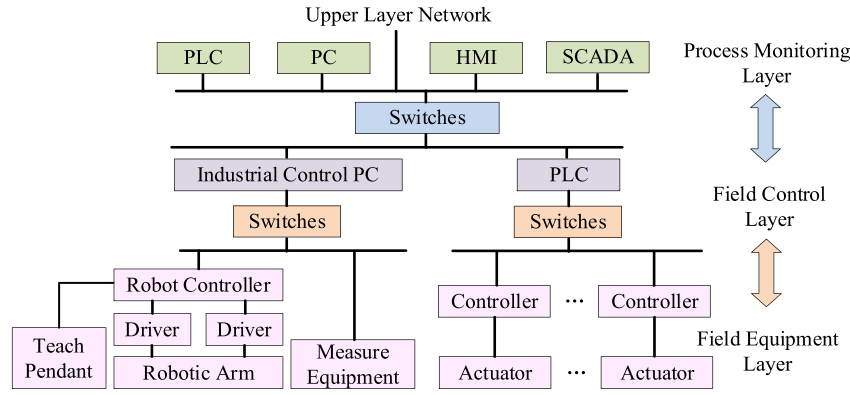


Fig. 1. Key equipment and communication relationship of IRS.

In large-scale IRS with multiple robots, the main controller (industrial control PC, PLC, etc.) realizes system-level business logic control.

The introduction of industrial internet technology has significantly increased the number and types of potential risks in IRS. Network connections, application development, and the integration of process accessories have all become sources of system risks. Exploitation of these risks can lead to inaccurate production results, process interruptions, physical damage to equipment, and loss of data confidentiality. The information security risks of IRS have the following characteristics:

- (1) Real-time nature. IRS often require real-time responses. Delayed execution of control commands can result in erroneous execution of system operations or even system crashes. Potential risks within the system can occur and spread rapidly within a short period.
- (2) Extensiveness. Risks may originate from both internal and external sources of the industrial robot system. Attacks targeting these risks can exploit software vulnerabilities or physical interfaces of the robot system. For example, attackers can manipulate the robot's control logic by exploiting software vulnerabilities, causing the robot to perform incorrect operations.
- (3) Complexity. IRS consist of multiple subsystems, each containing devices such as sensors, actuators, and controllers. The interactions between subsystems and devices add to the complexity of risk exploitation. Moreover, exploiting one risk may trigger others; for instance, a sensor failure might halt the entire production line.
- (4) Concealment. Most vulnerabilities associated with industrial robots are related to software and firmware. Exploiting code defects can disrupt functionality, and such malicious activities are often difficult to detect (Cottrell et al., 2021).

3.2. Structured risk data modeling

Before constructing an attack graph for IRS, it is crucial to perform data modeling of the potential risks within the system to abstract risk entities and extract their characteristics. A technique closely related to risk data modeling is threat modeling, which enables security analysts to identify potential attack processes and security vulnerabilities in a structured manner, and to quantify the severity of these threats. Various threat modeling methods have gained widespread recognition, including PASTA, STRIDE, and CORAS (Khan et al., 2017; Lund et al., 2011; Pape and Mansour, 2024).

According to ISO/IEC 13,335-1: Concepts and Models for Information and Communications Technology Security Management, risk has a broader connotation than threat, encompassing elements such as assets and vulnerabilities in addition to threats. In the specific modeling process, risk data modeling focuses more on the data structure of risk entities, rather than the description of attack behaviors found in threat

modeling. This modeling approach offers greater scalability in environments with complex attack processes and diverse attack types. A risk data model (RDM) comprises the following components:

- (1) **Basic Risk Elements.** Risk refers to the probability of asset loss due to the exploitation of vulnerabilities by threats. For a specific risk object r , its corresponding asset $r.As$, threat $r.Th$, and vulnerability $r.Vu$ can provide a foundational description of the risk characteristics. For example, a hacker exploits a software vulnerability in an industrial robot system through a man-in-the-middle attack to compromise the confidentiality of sensitive data.
- (2) **Attack Behavior and System Resources.** Basic risk elements represent the static attributes of a risk, while an attack graph describes the process of an attacker executing risk exploitation. Therefore, $r.At$ is defined to represent the attack against r . In the attribute attack graph, nodes include not only atomic attacks but also nodes describing system attributes. $r.At.Re_F$ and $r.At.Re_B$ is defined to represent the preconditions required for $r.At$ and the resources or capabilities obtained after a successful attack, respectively.
- (3) **Risk Linkage Relationships.** For risk objects r_i and r_j , a linkage relationship $\lambda = (r_i, r_j)$ is defined to indicate a relationship of exploitation between the risks. If the linkage relationship λ and one risk object r_i are known, the other risk object r_j can be identified, as

$$r_j = \psi(r_i, \lambda), \quad (1)$$

At the class level, for risk classes R_i , R_j , and the linkage relationship class $\Lambda = \{\lambda_1, \lambda_2, \dots, \lambda_m\}$, there is

$$R_j = \Psi(R_i, \Lambda), \quad (2)$$

The industrial robot system attack graph describes the linkages between attacks using directed edges, denoted as

$$e = (r_i.At_p, r_j.At_q) | r_j = \psi(r_i, \lambda) \vee r_i, \quad (3)$$

- (4) **Probability of Successful Risk Exploitation and Exploitation Logic.** When using the attack graph for system risk assessment, it is necessary to convert the elements of the attack graph into probabilities. For the risk exploitation step $r.At$, an associated time probability distribution can provide a preliminary evaluation of the likelihood of successful risk exploitation, denoted as

$$\phi(r.At) = P(T_1(r.At) = t), \quad (4)$$

There are two basic logical relationships for risk exploitation: $\sigma(r.At) \in \{\&, |\}$. If

$$\begin{cases} \sigma(r_k.Ats) = \& \\ e_i = (r_i.Atp, r_k.Ats) \in \mathbf{E}, \\ e_j = (r_j.Atq, r_k.Ats) \in \mathbf{E} \end{cases} \quad (5)$$

then the successful execution of $r_k.Ats$ requires the prior successful execution of both $r_i.Atp$ and $r_j.Atq$. In this case, the global success probability of $r_k.Ats$ is

$$\begin{cases} \phi(r_k.Ats) = P(T_g(r_k.Ats) = t) \\ T_g(r_k.Ats) \geq T_1(r_i.Atp) + T_1(r_j.Atq) + T_1(r_k.Ats) \end{cases} \quad (6)$$

To design the Risk Data Model (RDM), we refer to the MAL (Meta Attack Language) framework used in the threat modeling process (Widel et al., 2023). The risk class can be defined as

```
class RiskClass {
    BasicElement = ('Asset', 'Threat', 'Vulnerability')
    | Risk.AttackStep1[Gamma(α, β)] -> Risk.AttackStep2
    Risk.AttackStep1.ForwardResources = [res1]
    Risk.AttackStep1.BackwardResources = [res2]
}
```

The relationships between risks or associated assets are defined as

```
associations {
    RiskObject1 * (- RelationType -) * RiskObject2
    Risk.Asset 1 (- RelationType -) * Channel
}
```

Using the above syntax, the formal construction of the RDM can be achieved, providing a formal method for generating the industrial robot system attack graph.

3.3. Attack graph: modeling risk exploitation relationships

With the increasing complexity of systems, the number of state nodes in a state attack graph grows rapidly, leading to the problem of state space explosion. Additionally, state attack graphs lack intuitive advantages. In contrast, attribute attack graphs use atomic attacks and system resources as independent attribute nodes, offering better adaptability in large-scale networks.

Definition 1. An attribute attack graph is a directed acyclic graph (DAG) defined as a four-tuple $AG = (\mathbf{S}, \mathbf{A}, \mathbf{E}, \sigma)$

- (1) $\mathbf{S} = \{s_i | i = 1, 2, \dots, n\}$ is a set of attribute nodes. This variable represents the resources obtained by the attacker during the attack process. The set of attribute nodes can be further divided into three subsets based on the attacker's state $\mathbf{S} = \{S_i | S_i \in \mathbf{S}_{ext} \cup \mathbf{S}_{int} \cup \mathbf{S}_{ter}\}$, where $\mathbf{S}_{ext}$ represents the initial attribute nodes set, $\mathbf{S}_{int}$ represents the attribute nodes set during the attack process, and $\mathbf{S}_{ter}$ represents the attribute nodes set for the attacker's final targets. The attribute node values are either 1 or 0, indicating whether the attacker possesses access rights or other resources.
- (2) $\mathbf{A} = \{a_i | i = 1, 2, \dots, n\}$ is a set of atomic attack nodes. Atomic attacks represent the minimal set of operations required for an attacker to transition from one attribute node to another. After obtaining the resources of an attribute node, the attacker can achieve state transitions through atomic attack nodes. The values of atomic attack nodes are either 1 or 0, indicating whether the attack behavior has occurred or not.
- (3) $\mathbf{E} = \{E_1 \cup E_2\}$ is a set of directed edges in the attack graph. The directed edges are categorized based on the types of nodes they connect. E_1 indicates that the attacker initiates an attack by occupying relevant resources, and E_2 indicates that the attack behavior enables the attacker to acquire new resources.
- (4) σ is a set of relationship types between atomic attack nodes and their preceding attribute nodes. For an atomic attack node a ,

complex resource utilization relationships may exist to execute the attack behavior corresponding to a . Depending on the network characteristics and the resources obtained through the attack, the preceding nodes are connected to the atomic attack node using "AND," "OR," or a combination of both logic types.

Attribute-based attack graphs based on DAGs play a significant role in modeling risk exploitation relationships, offering notable advantages. The structural properties of a DAG ensure the acyclic nature of attack paths, allowing risk exploitation relationships to be represented in an intuitive and systematic manner. In this model, the nodes of the DAG represent system resources, attack behaviors, and associated risk attributes, while the edges indicate the relationships and dependencies between these nodes. By defining such a DAG-based attack graph, potential attack paths can be more clearly depicted, enabling an analysis of their impact on system security. This structured representation allows FuzzAGG to generate attack graphs for IRS more efficiently and evaluate the validity of various attack paths. Furthermore, the use of DAG-based attribute attack graphs enhances scalability, optimizing the attack graph generation process within the FuzzAGG framework to better address the security challenges posed by complex IRS. This approach enables more flexible adaptation to evolving risk environments.

4. Fuzzing-Driven attack graph generation for IRS

This section discusses the process of constructing an attack graph for IRS. Generating an attack graph requires not only a pre-constructed database of industrial robot system vulnerabilities but also knowledge of the network topology, configuration information, and potential risk exploitation relationships of the industrial robot system. However, this information is often difficult to obtain directly, as it cannot be easily sourced from public data or expert experience. As the scale of attack graph nodes increases, more computational resources are needed to analyze the relationships between nodes. Therefore, this section introduces a fuzz testing mechanism to construct test cases aimed at the risk exploitation process of IRS, thereby enabling the efficient generation of attack graphs.

4.1. Attack graph generation framework

The proposed FuzzAGG attack graph generation framework is illustrated in Fig. 2, comprising three units and two checkers. The components of FuzzAGG are detailed below.

- (1) **Fuzz Testing Unit.** The fuzz testing unit is one of the core components of the attack graph generation framework. It takes industrial robot system description files as input and uses built-in fuzz testing algorithms to generate structured risk data model object instances. The results output by the fuzz testing unit can be represented as specific risk data model objects, as described in Section 3.2.

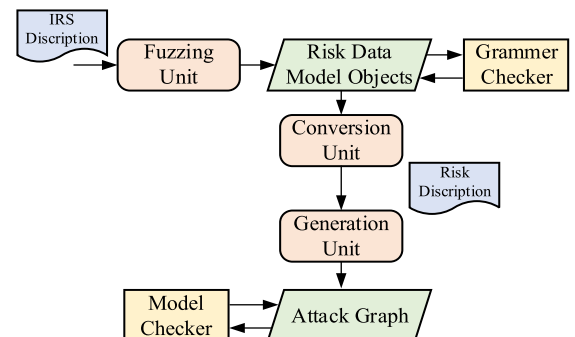


Fig. 2. FuzzAGG attack graph generation framework.

- (2) **Conversion Unit.** The conversion unit integrates and consolidates the risk data model objects (RDMOs) generated by the fuzz testing unit into risk description files.
- (3) **Generation Unit.** The generation unit is another core component of the attack graph generation framework. It aims to further construct the risk description files into a graphical attack graph. The generation unit performs the following tasks: 1) Remove redundant risk attributes. 2) Graphically display the relationships between nodes.
- (4) **Syntax Checker.** Since the risk data model is structured, the risk data model objects generated by the fuzz testing unit can be validated. In cases where data is insufficient, some risk attributes in the risk data model objects might be missing. The syntax checker ensures that incomplete risk data model objects still meet the structured format requirements necessary for the generation of the risk attack graph.
- (5) **Model Checker.** When the generation unit produces and graphically displays the industrial robot system attack graph, the model checker examines issues such as node reachability and loops in the attack graph to ensure the usability of the generated attack graph.

4.2. Risk data model objects and attack graph generation

The generation of risk data model objects and attack graphs are key tasks of the proposed FuzzAGG framework. Therefore, the design of the fuzz testing unit and the generation unit requires particular attention. The completeness and accuracy of risk data model objects, serving as prior knowledge for the attack graph, are crucial for the efficient and precise generation of the attack graph. Although Section 3.2 provides a structured outline of the risk data model objects, in IRS, the specialized nature of equipment, software, and services compared to general information systems means that the directly obtained risk data is often unstructured or even incomplete. Therefore, to extract the necessary descriptive elements for risk data model objects, it is essential to analyze the specific risk characteristics of IRS through various testing methods, based on a deep understanding of the system.

A finite state machine (FSM) is a formal model that can be used to describe system states. By employing FSM and its time extension strategies, it is possible to model the states of an industrial robot system. Under ideal operating conditions, an FSM can describe all safe states of an industrial robot system and the constraints required for state transitions. However, risk data model objects focus on the risk attributes of IRS, and the ideal FSM lacks risk states. Therefore, the process of generating risk data model objects in IRS is illustrated in Fig. 3. First, the abnormal input points of the industrial robot system are identified. Then, the ideal FSM model is improved to form a test FSM model (TFSM), and test state sequences are extracted. Next, risk state test cases are generated and input into the ideal FSM. Based on the test results, risk attributes are extracted and eventually integrated into the risk data model objects.

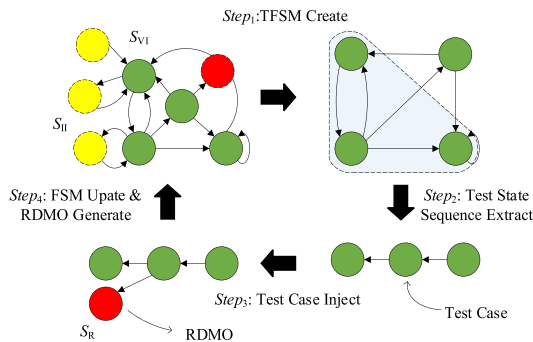


Fig. 3. IRS risk data model object generation.

For a FSM model that describes the complete operational mechanism of an industrial robot system and the potential risk exploitation, there are three types of states: Invalid Input State S_{II} (where the device cannot receive external input), Valid Input State S_{VI} (where the device can receive external input), and Risk State S_R (a new state triggered by input test cases). To improve the coverage rate of transitions from S_{VI} to S_R in the FSM model, this paper introduces a fuzz testing strategy for generating test cases for the FSM model. Fuzzing, as an automated testing method, is often used to discover defects and security vulnerabilities in the tested object. It can automatically generate a large amount of abnormal, random, or semi-random data and input it into the test target, thereby observing the target behavior and output, as well as triggering anomalies. When implementing fuzzing on the FSM model, state nodes provide corresponding feedback for each test case. For example, in the communication process, if the test case cannot be correctly received and processed, the device will inform the test-initiating device through a message. Regarding the fuzz testing of the TFSM model, test cases can be classified into the following four categories based on their effects after being output: (1) Normal Test Case (TC_N). The TFSM correctly receives and processes this type of test case, leading to the expected state transition. (2) Error Test Case (TC_E). The TFSM cannot receive and process this type of test case, and no state abnormal transition occurs. (3) Abnormal Test Case (TC_A). The TFSM can receive and process this type of test case, but it triggers an unexpected state transition. (4) Illegal Test Case (TC_I). The TFSM can receive and process this type of test case, but it cannot transition to a known state.

To illustrate the differences between these four types of test cases, we first define the FSM/TFSM model:

Definition 2. The FSM/TFSM is a five-tuple, denoted as $FSM = (S; I; O; \delta, \gamma)$, where:

- (1) S is the set of states of the FSM/TFSM, with $s_0 \in S$ being the initial state of the FSM.
- (2) I and O are the input and output sets of the FSM/TFSM.
- (3) δ is the state transition function of the FSM/TFSM, represented as $\delta: S \times I \rightarrow S$.
- (4) γ is the output function of the FSM/TFSM, represented as $\gamma: S \times I \rightarrow O$.

To validate the effectiveness of the FuzzAGG framework in risk identification and attack graph generation for IRS and its associated risk's FSM model, this paper presents several typical adversarial scenarios. These scenarios effectively simulate common security threats in IRS and reveal the system's risk behaviors under abnormal conditions. The scenarios include, but are not limited to: 1) exploiting communication protocol vulnerabilities to manipulate the robot controller and induce abnormal operations; 2) performing a man-in-the-middle attack to gain unauthorized control over the PLC and actuators; and 3) injecting erroneous driver state data to mislead the HMI display and disrupt PLC logic control. These scenarios can be implemented by simulating interactions between state information and control commands within the FSM. The FSM model accurately depicts the state changes and data flow in the IRS, allowing for the simulation of intrusion behaviors by inserting abnormal inputs or manipulating communication protocols during state transitions. Utilizing fuzz testing techniques, the FuzzAGG framework can automatically generate a large number of diverse anomalous test cases, explore potential attack paths, and readily trigger these risk exploitation scenarios.

Fig. 4 shows two FSM models, one for the driver and one for the attacker. The messages sent and received by the driver adhere to a certain fixed protocol. When the attacker sends a message that does not conform to the protocol (TC_E) to the driver, the driver recognizes and discards the message. Suppose the correct messages received by the driver from other system devices are periodic. If the attacker knows the message protocol and sends messages to the driver according to the correct message periodicity (TC_N), the driver transitions to the expected

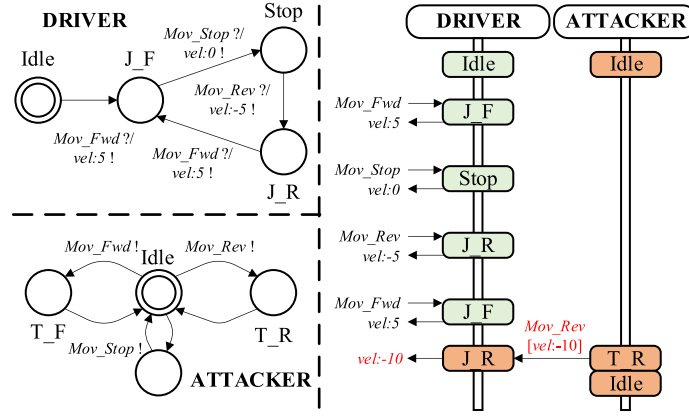


Fig. 4. Examples of state models and their messaging.

state. In Fig. 4, the attacker enters the T_R state and sends the message *Mov_Rev* to the driver, which is in the J_F state. According to the FSM description, the next state of the driver under normal circumstances is Stop. However, the output message from the attacker's state machine (TC_A) causes the J_F state to unexpectedly jump to J_R, resulting in an error in the system's business logic. For incomplete or coarse-grained state machine models, the attacker can send specifically crafted messages (TC_I) to transition the FSM to unknown states, such as shutdown states or error states.

The goal of fuzz testing on the FSM is to identify test cases that can cause the state machine to undergo abnormal state transitions and produce unexpected outputs. To enhance the quality of generated test cases, it is important to increase the occurrence rate of TC_A and TC_I during the generation process. Given that fuzz testing for state machines needs to adapt to complex input structures and explore new state transition paths while ensuring high flexibility and scalability of the generation rules, this paper proposes the use of a strategy-based fuzz testing engine. Since the generation of test cases is closely related to the state sequences within the TFMS model and the input sequences corresponding to these states, a TFMS fuzz test case generation framework based on the MU-SeqGAN model is proposed. The training process of this model is illustrated in Fig. 5. The state sequences and corresponding input sequences from the TFMS network composed of industrial robot system assets are selected and encoded. The encoded input sequences $\{E(i)|i \in I\}$ serve as real data, while the encoded state sequences $\{E(s)|s \in S\}$ are input into the generator (RNN) to produce encoded input sequences $\{E(i)|i \in I\}$ as generated data. The MU-SeqGAN model is based on the SeqGAN model, utilizing a discriminator (CNN) and Monte Carlo tree search to calculate the value of input behaviors and achieve discriminator training through policy gradient updates. Building on SeqGAN, MU-SeqGAN concatenates the generated $\{E(i)|i \in I\}$ and $\{E(s)|s \in S\}$, decodes them into generated test cases $\{(s, i)|s \in S,$

$i \in I\}$, and evaluates their effectiveness by monitoring the state changes and outputs of the TFMS network, updating the TFMS network, and training the generator accordingly.

Based on the test case generation model training process shown in Fig. 5, this paper proposes a method for generating RDMOs, as depicted in Fig. 6.

- (1) Data Preprocessing Stage. In this stage, the state set and input set of the industrial robot system's TFMS are processed through the encoding unit. Elements within the state set are encoded using Gray code, discrete parameters in the input set are encoded using One-Hot encoding, and continuous parameters are first binned and then One-Hot encoded. The encoded data is stored in the state encoding database and the associated input encoding database. The TFMS trained by the MU-SeqGAN model adds new state sets S_r as risk exploitation states and O_r as risk exploitation outputs.
- (2) Test Case Generation Stage: Encoded state sequences that match the state distribution characteristics within the TFMS are selected from the state encoding database $\{s'_g = E(s)|s \in S\}$. MU-SeqGAN generates encoded input sequences $\{i'_g = E(i)|i \in I\}$. These two encoded sequences are concatenated and input into the decoding unit to obtain test cases for the industrial robot system's TFMS.
- (3) RDMOs Generation Stage: The generated test cases are input into the TFMS model and executed within the FSM simulator environment. The transition states S_t and their outputs O_t during the TFMS execution process are continuously monitored, and test cases are classified based on the results. If effective test cases (TC_A , TC_I) are found, abnormal state transitions and outputs within the TFMS are extracted. The mapping relationship between TFMS elements and RDMO elements is used to automatically generate RDMOs.

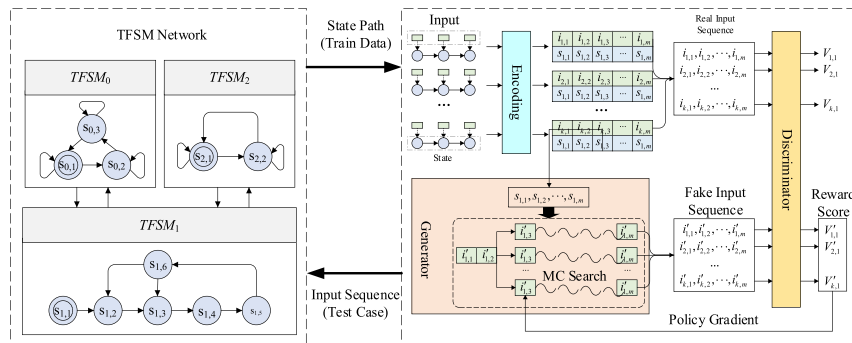


Fig. 5. Training process of TFMS fuzzing test case generation based on MU-SeqGAN.

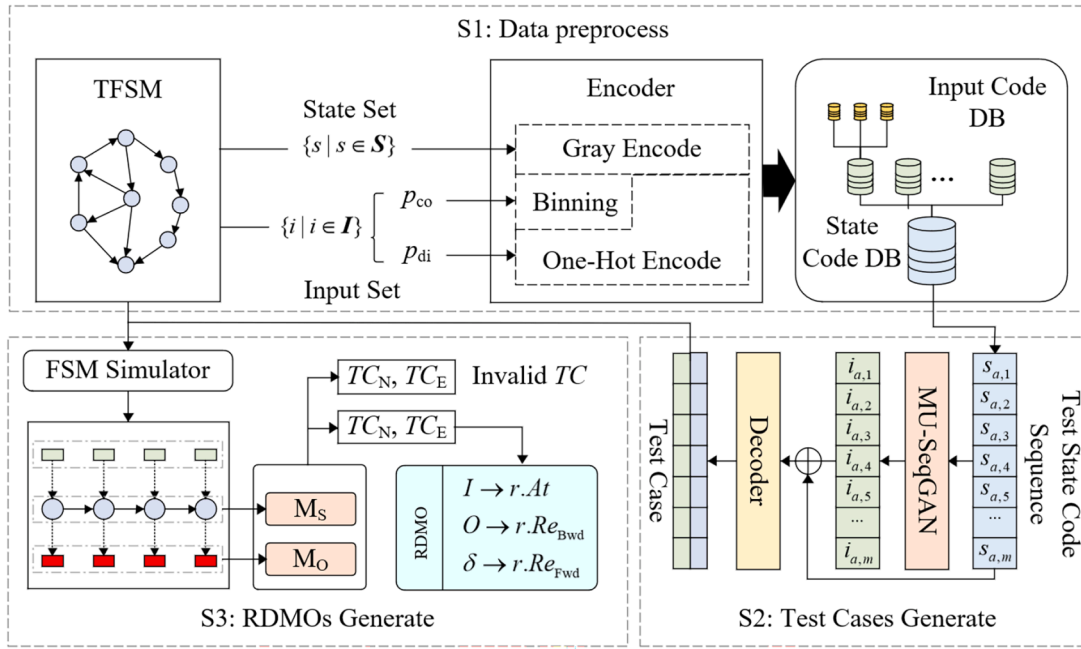


Fig. 6. Risk data model objects generation.

Once RDMOs within the IRS's TFSM have been obtained using [Algorithm 1](#) automatic generation of the industrial robot system's attack graph can be realized. According to the definition of the attribute attack graph described in [Definition 1](#), the attack graph should include attribute nodes, atomic attack nodes, and directed edges linking these nodes. Since each RDMO contains descriptions of specific risk exploitation processes, including atomic attacks and related resource utilization relationships, generating a single RDMO attack graph can be easily achieved. To generate multiple RDMO attack graphs, it's necessary to consider the linkage between risks corresponding to each RDMO. To enhance the efficiency of attack graph generation, firstly, for each RDMO, the linkage relationship between atomic attacks $r_i.At_i$, $r.At.Re_F$ and $r.At.Re_B$ can be identified to obtain the linkage relationship between atomic attack nodes and attribute nodes within the RDMO. Then, based on the containment relationship between the initial attribute nodes and the final target attribute nodes within each RDMO, the relationship between RDMOs can be determined. [Algorithm 2](#) outlines the process of generating attack graphs based on RDMOs.

4.3. Risk data model objects and attack graph verification

After generating the RDMOs for the industrial robot system and the corresponding attack graphs using the method proposed in [Section 4.2](#),

Algorithm 1

RDMO Generation.

Input: TFSM(S, I, O, δ, γ)

$S' \leftarrow s' = E_G(s)$
 $I' \leftarrow i' = \{p'_{co}, p'_{di}\} = \{E_B(p_{co}), E_O(p_{di})\}$
loop:
 $\text{select } S'_g \subseteq S'$
 $I'_g = \text{MU-SeqGAN}(S'_g)$
 $TC = (S'_g, I'_g) \leftarrow S'_g, I'_g = D(S'_g \subseteq S', i'_g \in I'_g)$
 $S_t, O_t, \delta_t = \text{TFSM.Simu}(I'_g)$
 if $s_t \in S_t$ **or** $o_t \in O_t$:
 $TC_t \leftarrow TC$
 $r.At = I'_g, \sigma = I'_g(p_{co}, p_{di})$,
 $r.At.Re_B = O_t, r.At.Re_F = \delta_t$,
 RDMO $\leftarrow (r.At, r.At.Re_B, r.At.Re_F, \sigma)$
return RDMO

it is necessary to perform formal verification to determine if there are any syntax errors in the RDMOs or if there are any unreachable nodes in the attack graphs. This verification work needs to be executed immediately after the objects to be verified are generated.

Industrial robot system RDMOs based on RDM need to be checked for two properties: (1) Syntax compliance: Since RDMOs use the modeling language RDM, which is proposed based on the domain-specific language MAL, RDMOs should adhere to the strict syntax specifications of MAL, and the extension part of risk descriptions should also be consistent with the basic syntax of MAL. (2) Attack graph generation feasibility: RDMOs are an intermediate link in building industrial robot system attack graphs. Therefore, RDMOs generated based on the MU-SeqGAN model's TFSM fuzzy test cases should contain attributes that can satisfy the generation of attack graphs.

To achieve automated verification of syntax compliance, a syntax checking tool can be used to ensure that RDMOs follow the defined syntax rules. ANTLR (ANother Tool for Language Recognition), a powerful parser generator, can generate lexical analyzers, parsers, etc., from formal grammar descriptions. It is widely used in domain-specific language processing, and the latest version is ANTLR 4. In the specific syntax checking process, a .g4 file is created to define lexical rules to recognize keywords, identifiers, etc., in RDM, and parsing rules are defined to build RDM syntax structures such as expressions, statements,

Algorithm 2

AG Generation.

Input: $RDMO(R)$

```

for i in range (len(R)):
  for j in range len( $r_i.At$ ):
     $E \leftarrow e(r_i.At_j.Re_F, r_i.At_j), e(r_i.At_j, r_i.At_j.Re_B)$ 
    if  $\sigma(r_i.At_j) = \&$ :
       $e(r_i.At_j.Re_F, r_i.At_j).attr = \text{'AND'}$ 
    else:  $e(r_i.At_j.Re_F, r_i.At_j).attr = \text{'OR'}$ 
    for k in range (len(R)):
      if  $\min(r_k.At.Re_F) \text{ in } \max(r_i.At.Re_B)$ :
         $E \leftarrow e(r_i.At.Re_B, r_k.At_j)$ 
         $r_k.At.Re_F.del()$ 
   $AG \leftarrow r.At.Re_F, r.At, r.At.Re_B, E$ 
return AG

```

etc. Running the .g4 file generates code for lexical analyzers and parsers, which recognize input RDMOs and create listener classes to traverse the abstract syntax tree generated by ANTLR, ultimately determining whether there are syntax errors in the RDMOs.

For the feasibility of generating attack graphs from RDMOs, Definition 1 provides the characteristics required for constructing attack graphs, including four types of attributes: attribute nodes, atomic attack nodes, directed edges, and relationship types of nodes. According to the characteristics of attribute attack graphs, atomic attack nodes need to be linked to their preceding and subsequent attribute nodes. Therefore, each $r.At$ in RDMO should define at least $r.At.Re_F$ or $r.At.Re_B$. In the case where atomic attack nodes only contain one type of attribute node, missing attribute nodes can be supplemented by attribute nodes of other atomic attack nodes with linkage relationships.

$$\begin{cases} r_i.At_j.Re_F \leftarrow r_i.At_k.Re_B | e = (r_i.At_k, r_i.At_j) \\ r_i.At_j.Re_B \leftarrow r_i.At_k.Re_F | e = (r_i.At_j, r_i.At_k) \end{cases} \quad (7)$$

The attack graph generated from RDMO needs to be verified for reachability. Based on the type of nodes, this can be further divided into the reachability of atomic attack nodes and the reachability of attribute nodes, indicating whether the attacker can achieve specified attack behaviors or obtain specified system attributes through the attack graph. This paper proposes an attack graph reachability verification algorithm for analyzing sub-attack graphs containing only one type of node using breadth-first search algorithm, as shown in Algorithm 3. After verifying the reachability of all nodes in the attack graph, software such as Graphviz can be used for graphical visualization to draw the attack graph of the industrial robot system.

4.4. Performance analysis and comparison with existing methods

This section presents a performance analysis of the proposed FuzzAGG framework, focusing on the time complexity and coverage rate of risk-triggering test cases. Additionally, a comparison is made with existing studies across several key functionalities. Algorithms 1, 2, and 3

Algorithm 3

AG reachability verification.

Input: AG

```

SQ, AQ = new query(), SV, AV = new nodeset()
AG.S = AG.del(A), AG.A = AG.del(S)
AQ.push(A0), SQ.push(S0)
while SQ, AQ not null:
  s_u, a_u ← SQ.pop(), AQ.pop()
  if s_u == st, a_u == at:
    return 'st, at is reachable'
  if s_u, a_u not in SV, AV:
    SV.add(s_u), AV.add(a_u)
    for s_v, a_v in AG.S.nn(s_u), AG.A.nn(a_u):
      if s_v, a_v not in SV, AV:
        AQ.push(s_v), SQ.push(a_v)
return 'AG is not reachable'

```

address the generation of Risk Data Model Objects (RDMOs), attack graph construction, and attack graph verification, respectively, within the FuzzAGG framework. Algorithm 1 generates RDMOs by monitoring whether the Industrial Robot System (IRS) state model enters an abnormal state in response to input test cases. Factors such as the size of the state space and the number of risk states in the IRS model affect the algorithm's runtime. To evaluate performance, a state model with a simple and adjustable topology was constructed.

Algorithm 1 identifies risk state transitions by exploiting design flaws in the constraints of normal state node transitions and conducts deep mining of RDMOs according to the state transition sequence. The performance testing of Algorithm 1 examined the effects of the ratio of risk states (N_{RS}) to normal states (N_{NS}) and the depth of the state sequence, denoted as d . The results, shown in Fig. 7, indicate that the N_{RS}/N_{NS} ratio has a minimal impact on the execution time of Algorithm 1. The primary factor causing a gradual increase in runtime is the relatively time-consuming process of assembling the identified risk elements into RDMOs. A linear positive correlation between the state sequence depth and the execution time of Algorithm 1 is evident in Fig. 7. This result is expected, as longer sequence lengths involve more states and

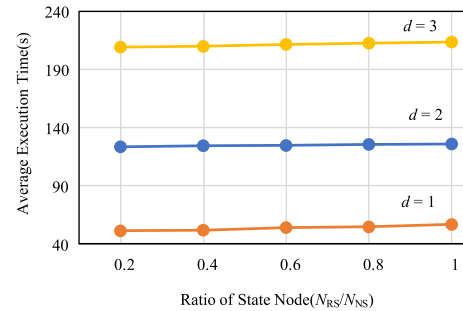


Fig. 7. Impact of N_{RS}/N_{NS} , d on the operational efficiency of algorithm 1.

transitions, thereby increasing the number of computational steps required. Given that the operations to trigger risk transitions for similar states are essentially the same, the observed results align well with theoretical expectations.

Algorithm 1 was tested for its efficiency and risk state coverage across different sizes of state spaces, with the ratio of N_{RS}/N_{NS} and the state sequence depth d fixed at 0.2 and 1, respectively. The results are shown in **Table 1**, where the runtime exhibits an approximately linear positive correlation with the size of the state space. In contrast, the coverage of risk states decreases as the state space expands, eventually stabilizing at around 88 %. Analyzing the state model's transition constraints reveals that the uncovered risk states are primarily associated with transitions between risk states. **Table 2** presents the performance of **Algorithms 2** and **3**. Since these algorithms take structured RDMOs and the graph-based attack graph as input, their efficiency is significantly higher than that of **Algorithm 1**. Additionally, both algorithms maintain a linear positive correlation with the size of the state space, indicating strong performance in generating and validating large-scale attack graphs.

This paper compares the proposed method with other attack graph generation approaches using five criteria: state-driven, dynamic adaptability, risk identification, scalability, and model checking. The rationale for choosing these criteria is as follows:

- 1) State-driven: IRS exhibits time-sensitive and cyclic operational characteristics. State-driven attack graph generation can effectively address the complex and varying operational conditions in IRS.
- 2) Dynamic Adaptability: The system configuration and potential threats in IRS change over time, and the attack graph should dynamically adjust to reflect new risks.
- 3) Risk Identification: During the generation process, the attack graph should be capable of identifying potential unknown attacks, thereby providing recommendations for security defense.
- 4) Scalability: The attack graph should adapt to complexity and large-scale environments, maintaining generation efficiency as the number of nodes increases.
- 5) Model Checking: The attack graph should be verified for correctness and reachability to ensure it accurately reflects the system's risk status.

Based on these criteria, a comparison with methods introduced in the literature is presented in **Table 3**. In the table, a checkmark or a cross indicates whether an attack graph generation method possesses the corresponding characteristic. The comparison reveals that FuzzAGG has advantages in state-driven generation, risk identification, and model checking, making it more suitable for the automated generation of attack graphs for IRS state models with incomplete risk data.

Table 1
Operational efficiency and risk state coverage of RDMO generation process.

Input Size(N)	Average Execution Time(s)	Risk state coverage(%)
100	258.3	0.99
200	397.7	0.96
300	585.5	0.94
400	814	0.93
500	990.4	0.91
600	1245.6	0.89
700	1555.2	0.89
800	1904.2	0.88
900	2073.9	0.88
1000	2483.4	0.88

Table 2
Operational Efficiency of Attack Graph Generation and Verification Process.

Input Size(N)	AG Generation Average Execution Time(s)	AG Reachability Verification Average Execution Time(ms)
100	0.97	1.96
200	1.67	3.12
300	2.46	5.19
400	3.95	7.55
500	4.8	8.36
600	5.93	9.9
700	6.46	12.24
800	7.22	13.57
900	8.57	15.28
1000	9.59	16.7

Table 3
Comparison of Features of Attack Graph Generation Methods.

Ref	State-Driven	Dynamic	Risk Identification	Scalable	Model Checking
(Chen et al., 2009)	×	✓	×	✓	×
(Al Ghazo et al., 2019)	×	✓	×	×	✓
(Koo et al., 2022)	×	×	×	✓	×
(Mohammadzad et al., 2023)	×	✓	✓	✓	×
FuzzAGG	✓	✓	✓	✓	✓

5. Simulation verification and analysis

5.1. Simulation environment configuration

To validate the applicability of the FuzzAGG framework for generating attack graphs of IRS, this paper selects the laboratory-built PCB automation production line industrial robot system as the target (**Fig. 8**). The system comprises a conveyor subsystem and a robotic arm monitoring subsystem, consisting of 1 motion controller, 6 SCARA robots, and various supporting peripheral devices, with a total of 6 operation stations and 2 material transfer stations. Each station is a relatively independent unit that can be freely split and reassembled.

The conveyor subsystem achieves motion control of the stations via EtherCAT. PLC reads sensor signals and controls I/O and drives based on the system's business logic program. HMI serves as a human-machine interaction device to view real-time station position information and manually control the conveyor motor speed. Drivers and I/O connect sensors and motors of the conveyor subsystem. The robotic arm monitoring subsystem monitors and controls the robotic arms of the 6 operation stations via Modbus TCP. The teach pendant serves as a human-machine interaction device, with manual mode for debugging robotic arm operation and automatic mode for executing business logic code. A switch connects the PC and robot controllers, forwarding Modbus TCP robot operation information and control commands, and sending robot status information to the PLC of the conveyor subsystem. There exists a business logic relationship between the conveyor subsystem and the robotic arm monitoring subsystem during operation. When the stations controlled by the drives and motors reach the working position, they stop running, and the robotic arms execute tasks along set codes along the trajectory, stopping work until the next station reaches the specified position.

As the FuzzAGG framework targets FSM generation of IRS, this paper constructs the FSM of the industrial robot system using the Simulink/Stateflow tool in MATLAB. **Fig. 9** shows the simplified TFSM of the PCB automation production line industrial robot system, including all control devices and human-machine interaction devices from **Fig. 8**, except for the switch. TFSM also includes motor, actuator, and robotic arm models

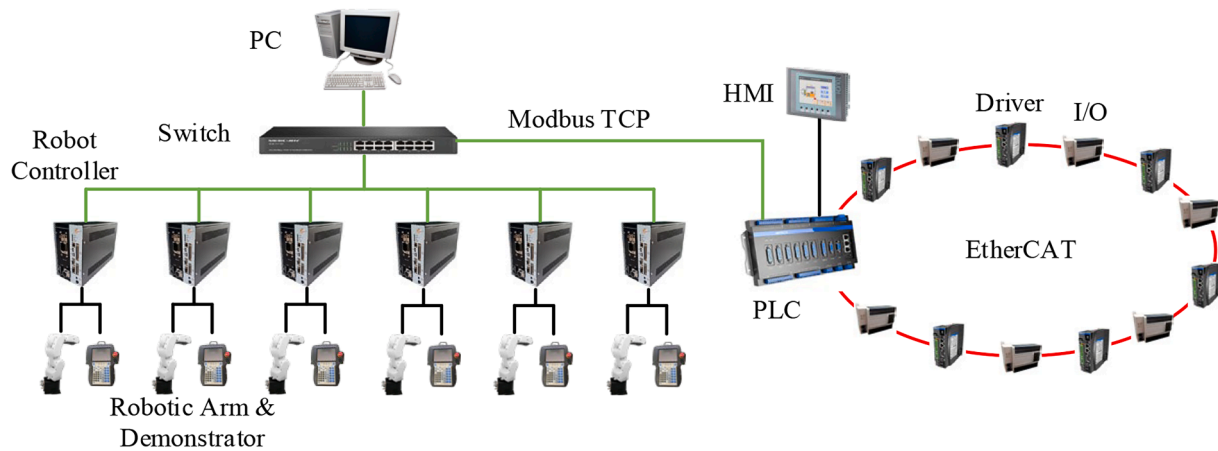


Fig. 8. PCB automation production line cyber-physical connection.

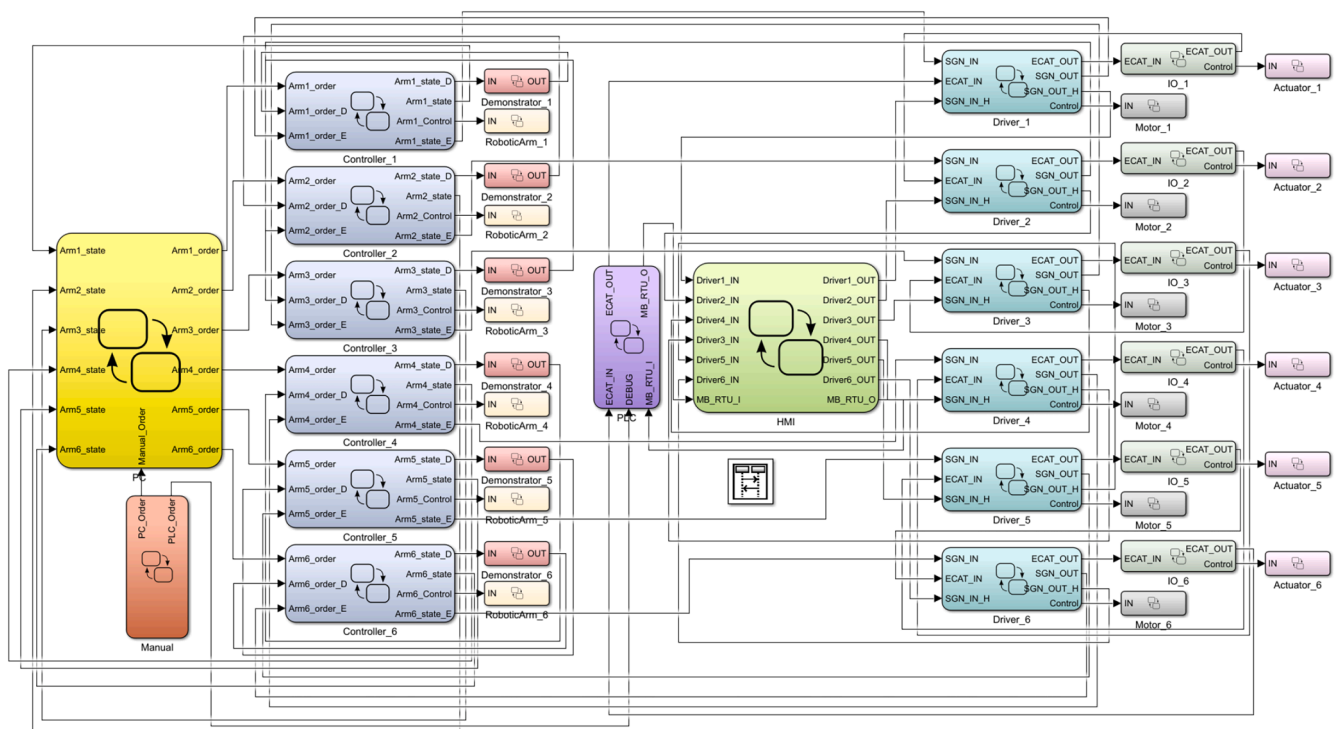


Fig. 9. Simulink/Stateflow-based TFSM for PCB automated production line.

to observe the response of TFSM after inputting the test cases generated by the FuzzAGG framework, and the Manual module represents the attacker's FSM, used to simulate the behavior and state description of the attacker sending fuzzy test cases to TFSM.

5.2. Simulation & analysis

Based on the FuzzAGG framework, fuzzy test case generation for the TFSM shown in Fig. 8 is performed, and the actions corresponding to the triggering of abnormal state transitions and abnormal outputs are transformed into attack state nodes and added to the attacker's FSM. The structure of the attacker's FSM is illustrated in Fig. 10. It can be observed that the final transition states of the attacker's FSM include `RoboticArm_{1:6}.mis`, `Motor_{1:6}.mis`, `Actuator_{1:6}.mis`, and `HMI.mis`, which respectively represent abnormal operations of the robotic arm, motor, actuator, and incorrect data display of the HMI. These states reach each other through directed edges with message-sending constraints between

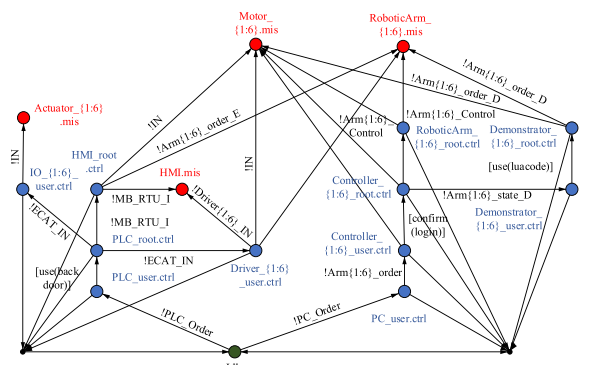


Fig. 10. Attacker FSM model for TFSM of PCB automation production line.

state nodes. Additionally, the connectivity in the model of Fig. 9 shows that the teach pendant forms a control loop with the controller and robotic arm, while the driver and motor form another control loop. In the attacker model presented in Fig. 8, the teach pendant, controller, and driver can influence other devices in the control loops by sending messages, aligning with the business logic relationships described in Section 5.1 involving the two subsystems.

Analyzing the attack states in the attacker model shown in Fig. 10 and extracting risk attributes, RDMOs corresponding to each attack state are constructed. Based on the syntax rules and format specifications of RDM, attack graphs are automatically generated using Algorithm 2. Considering the presence of multiple devices of the same model in the PCB automation production line industrial robot system, these devices are merged to simplify the structure of the generated attack graph, as shown in Fig. 11. The initial/final attribute nodes and atomic attack node sequences corresponding to the RDMO used to generate the simplified attack graph are listed in Table 4.

The initial attribute nodes of the attacker are user(0), msg(0,PC), and msg(0,PLC), indicating that the attacker possesses user privileges of device 0 (the computing device used for launching the attack) and communication capabilities with PC and PLC. The final attribute nodes of the attack graph are mis(RA1), mis(Mo1), mis(HMI), and mis(Ac1), consistent with the final transition states of the attacker's FSM shown in Fig. 10, confirming that the generated attack graph accurately describes the attacker's state transition relationships.

On a server with an AMD Ryzen 7 5800H 3.2 GHz processor, 16GB of RAM, and Windows 11, the FuzzAGG framework is executed for the TFSM of the PCB automation production line industrial robot system shown in Fig. 8. Based on this framework, 85 RDMOs are generated, along with an industrial robot system attack graph containing 184 attribute nodes and atomic attack nodes. The proposed Algorithm 1 runs in MATLAB R2024a for 473.2845s, Algorithm 2 for 1.5073 s, and Algorithm 3 for 0.0026s. These results demonstrate the effectiveness of the FuzzAGG framework, confirming its ability to generate attack graphs for IRS with complex network structures. Utilizing the generated attack graph of the industrial robot system, security analysts can allocate protective resources rationally, thereby enhancing the security of IRS at minimal protection costs.

The final attribute nodes in the attack graph, such as mis(RA1) and mis(Mo1), represent abnormal operations of key assets like robotic arms and motors within the IRS. These are potential risk exploitation targets in the system. A qualitative analysis of the features presented in Table 4 provides two important references for risk analysis: 1) Atomic Attack Node Sequence Length. This indicates the number of attack steps required to reach the final risk exploitation target from the initial state. Longer sequences generally mean that the attack path is more complex, requiring the attacker to navigate more steps, involve more system components, and achieve higher levels of privilege escalation. Such

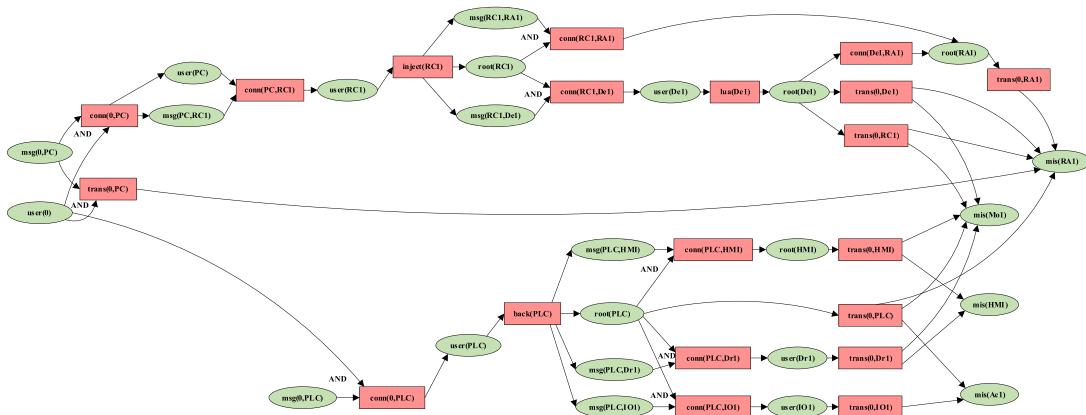
Table 4

Attack Paths Based on the Attack Graph Generated by FuzzAGG.

Initial Attribute Nodes	Atomic Attack Node Sequence	Final Attribute Nodes
user(0),msg(0,PC)	conn(0,PC)/conn(PC,RC1)/inject(RC1)/trans(0,RA1)	mis(RA1)
user(0),msg(0,PC)	conn(0,PC)/conn(PC,RC1)/inject(RC1)/conn(RC1,De1)/lua(De1)/conn(De1,RA1)/trans(0,RA1)	mis(RA1)
user(0),msg(0,PC)	conn(0,PC)/conn(PC,RC1)/inject(RC1)/conn(RC1,De1)/trans(0,De1)	mis(RA1)
user(0),msg(0,PC)	conn(0,PC)/conn(PC,RC1)/inject(RC1)/conn(RC1,De1)/lua(De1)/trans(0,Rc1)	mis(RA1)
user(0),msg(0,PC)	trans(0,PC)	mis(RA1)
user(0),msg(0,PLC)	conn(0,PLC)/back(PLC)/trans(0,PLC)	mis(RA1)
user(0),msg(0,PC)	conn(0,PC)/conn(PC,RC1)/inject(RC1)/conn(RC1,De1)/lua(De1)/trans(0,De1)	mis(Mo1)
user(0),msg(0,PC)	conn(0,PC)/conn(PC,RC1)/inject(RC1)/conn(RC1,De1)/lua(De1)/trans(0,Rc1)	mis(Mo1)
user(0),msg(0,PLC)	conn(0,PLC)/back(PLC)/conn(PLC,HMI)/trans(0,HMI)	mis(Mo1)
user(0),msg(0,PLC)	conn(0,PLC)/back(PLC)/trans(0,PLC)	mis(Mo1)
user(0),msg(0,PLC)	conn(0,PLC)/back(PLC)/conn(PLC,Dr1)/trans(0,Dr1)	mis(Mo1)
user(0),msg(0,PLC)	conn(0,PLC)/back(PLC)/conn(PLC,HMI)/trans(0,HMI)	mis(HMI)
user(0),msg(0,PLC)	conn(0,PLC)/back(PLC)/conn(PLC,Dr1)/trans(0,Dr1)	mis(HMI)
user(0),msg(0,PLC)	conn(0,PLC)/back(PLC)/trans(0,PLC)	mis(Ac1)
user(0),msg(0,PLC)	conn(0,PLC)/back(PLC)/conn(PLC,IO1)/trans(0,IO1)	mis(Ac1)

attacks are typically more challenging, demanding higher technical skills and resource investment. 2) Frequency of Final Attribute Nodes. This reflects the concentration of specific risk exploitation targets across all possible attack paths. The higher the frequency of a risk exploitation target, the more accessible it is along multiple attack paths, indicating a higher level of risk exposure in the system. The attribute node mis(RA1) appears frequently across various attack paths, with relatively short sequences, suggesting that the risk associated with the robotic arm is relatively easier to exploit. This implies that the robotic arm is a likely target for attackers, who can more easily manipulate the controller or send malicious commands to disrupt its operations. Conversely, the attribute node mis(Ac1) appears in fewer paths, indicating that attackers are less inclined to target the actuator.

For a more detailed quantitative risk analysis, historical data, expert opinion, or simulation results should be used to assign initial attack success probabilities to different nodes in the attack graph. These

**Fig. 11.** Simplified attack graph generation results for same device type merging.

probabilities can then be dynamically updated using Bayesian inference based on the graph structure and the conditional probabilities of nodes, ultimately providing a comprehensive risk assessment for individual components and the overall system.

To deploy and apply the proposed FuzzAGG framework in an IRS field setting, it is first necessary to comprehensively identify the device connectivity, communication mechanisms, and known potential risks associated with the communication processes. This information can be obtained from system documentation, captured network traffic, and device log files, and then translated into state nodes within the IRS state model. The transition constraints of the state nodes can be defined by analyzing the control logic of IRS devices (such as PLCs and robot controllers), parsing communication protocols, and performing fuzz testing in an isolated environment. Modeling tools like UPPAAL can be used to construct the Finite State Machine (FSM) of the IRS, enabling detailed modeling of each component's states and transition constraints. The MU-SeqGAN model is then employed to generate test cases targeting the FSM, which detect potential risk states and map them into Risk Data Model Objects (RDMOs). These RDMOs are converted into structured risk description files, from which the attack graph is automatically constructed. Reachability analysis and visualization are subsequently performed to help security analysts identify critical risk points and potential attack paths.

6. Conclusion

In this paper, we successfully develop and validate a new attack graph generation framework, FuzzAGG, specifically for industrial robot systems (IRS). This framework systematically describes IRS risk data through a Risk Data Model (RDM) constructed using the domain-specific language MAL. The core of the FuzzAGG framework lies in the test cases generated by the MU-SeqGAN model. These test cases are designed for the state machine model of IRS and are effectively mapped to Risk Data Model Objects (RDMOs), enabling the adaptive construction of attack graphs.

We utilize the Simulink/Stateflow tool to construct a finite state machine model of IRS for a PCB automated production line. Based on this model, the FuzzAGG framework is applied to automatically generate attack graphs, which are visually presented using the Graphviz tool. Additionally, we thoroughly analyze the runtime of the algorithms employed in the FuzzAGG framework and verify the reachability of the generated attack graphs using a breadth-first search algorithm, ensuring the accuracy and reliability of the generated results. Large-scale state-space tests show that the proposed FuzzAGG is able to complete the RDMO search and generation for 1000 state nodes in 2483.4 s, and complete the generation and validation of attack graphs of the corresponding scales in 9.59 s and 16.7 ms, respectively. The RDMO generation coverage can reach >88 %.

By executing FuzzAGG within the established runtime environment, we generated 85 RDMOs and constructed an IRS attack graph containing 184 attribute nodes and atomic attack nodes, which were realized within 474.7944s. The experimental results demonstrate that the FuzzAGG framework effectively generates attack graphs for IRSs with complex network structures. The generation of RDMOs and attack graphs, along with the runtime of the attack graph reachability validation algorithm, sufficiently showcase the efficiency of the FuzzAGG framework in automated attack graph generation.

The contributions of this paper are: 1) proposing a novel method for attack graph generation. 2) providing a practical tool for the security analysis of IRS. The attack graphs generated by FuzzAGG enable security analysts to allocate protection resources more rationally, thereby improving the security of IRSs with minimal protection costs.

Future research will focus on improving the coverage of fuzzy test cases for state machine models of IRS and studying information security risk assessment techniques based on attack graphs for IRS to further validate the effectiveness of the attack graph generation framework.

Abbreviation

Abbreviation	Full name
IRS	Industrial Robot System
RDM	Risk Data Model
RDMO	Risk Data Model Object
FSM	Finite State Machine
TFSM	Test Finite State Machine
TC	Test Case
MAL	Meta Attack Language
As	Asset
Th	Threat
Vu	Vulnerability
At	Attack
Re _F /Re _B	Forward/ Backward Resources
S/A	State/Attack Node Set
E	Edge Set
p	parameter
M _s /M _o	State/Output Monitor

CRediT authorship contribution statement

Xiaosheng Liu: Conceptualization. **Wenqi Jiang:** Methodology. **Zhongwei Li:** Methodology. **Xianji Jin:** Formal analysis. **Zihan Ma:** Validation. **Qingyang Li:** Writing – review & editing.

Declaration of competing interest

All authors disclosed no relevant relationships.

Acknowledgement

This research is supported by "Project for the Development and Application of Safety Testing and Verification Platform for Industrial Robots" under Grant TC200H01Q and Heilongjiang Touyan Innovation Team Program under Grant AUEA5640202520–09.

Data availability

Data will be made available on request.

References

- Al-Eiadeh, M.R., Abdallah, M., 2024. GeniGraph: a genetic-based novel security defense resource allocation method for interdependent systems modeled by attack graphs. *Comput. Secur.*, 103927.
- Al Ghazo, A.T., Ibrahim, M., Ren, H., Kumar, R., 2019. A2G2V: automatic attack graph generation and visualization and its applications to computer and SCADA networks. *IEEE Transact. Syst., Man, Cybernet.* 50, 3488–3498.
- Ammann, P., Wijesekera, D., Kaushik, S., 2002. Scalable, graph-based network vulnerability analysis. In: *Proceedings of the 9th ACM Conference on computer and communications security*, pp. 217–224.
- Barrère, M., Hankin, C., O'Reilly, D., 2023. Cyber-physical attack graphs (CPAGs): composable and scalable attack graphs for cyber-physical systems. *Comput. Secur.* 132, 103348.
- Beckers, K., Krautsevich, L., Yautsiukhin, A., 2014. Analysis of social engineering threats with attack graphs. *Internat. Workshop Data Privacy Manag.* 216–232.
- B. Bezawada, I. Ray, K. Tiwary, AGBuilder: an AI tool for automated attack graph building, analysis, and refinement, data and applications security and privacy XXXIII: 33rd Annual IFIP WG 11.3 Conference. (2019) 23–42.
- Bouou, A.L.P.T., Lienou, J.P., Nelson, F.F., Shetty, S., Geh, W.E., Kamhoua, C.A., 2023. Cyber resilience measurement through logical attack graph analysis. In: *ICC 2023-IEEE International conference on communications. IEEE*, pp. 6034–6039.
- Chen, F., Su, J., Zhang, Y., 2009. A scalable approach to full attack graphs generation, engineering secure software and systems: first international symposium ESSoS 2009, Leuven, Belgium, February 4–6, 2009. In: *Proceedings 1. Springer Berlin Heidelberg*, pp. 150–163.
- Cottrell, K., Bose, D.B., Shahriar, H., Rahman, A., 2021. An empirical study of vulnerabilities in robotics. In: *2021 IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC). IEEE*, pp. 735–744.
- Haque, S., Keffeler, M., Atkison, T., 2017. An evolutionary approach of attack graphs and attack trees: a survey of attack modelling. In: *Proceedings of the international conference on security and management (SAM), The steering committee of the world*

- congress in computer science, computer engineering and applied computing (WorldComp), pp. 224–229.
- Homer, J., Zhang, S., Ou, X., Schmidt, D., Du, Y., Rajagopalan, S.R., Singhal, A., 2013. Aggregating vulnerability metrics in enterprise networks using attack graphs. *J. Comput. Secur.* 21, 561–597.
- Ingols, K., Lippmann, R., Piwowarski, K., 2006. Practical attack graph generation for network defense. In: 2006 22nd Annual computer security applications conference (ACSAC'06). IEEE, pp. 121–130.
- Ivanova, M.G., Probst, C.W., Hansen, R.R., Kammüller, F., 2016. Transforming graphical system models to graphical attack models. In: *Graphical Models for Security: Second International Workshop (Verona)*, pp. 82–96.
- Khan, R., McLaughlin, K., Lavery, D., Sezer, S., 2017. STRIDE-based threat modeling for cyber-physical systems. In: 2017 IEEE PES innovative smart grid technologies conference Europe (ISGT-Europe). IEEE, pp. 1–6.
- Konsta, A.M., Lafuente, A.L., Spiga, B., Dragoni, N., 2024. Survey: automatic generation of attack trees and attack graphs. *Comput. Secur.* 137, 103602.
- Koo, K., Moon, D., Huh, J.H., Jung, S.H., Lee, H., 2022. Attack graph generation with machine learning for network security. *Electronics*. 11, 1332.
- Li, W., Vaughn, R.B., Dandass, Y.S., 2006. An approach to model network exploitations using exploitation graphs. *Simulation*. 82, 523–541.
- Liu, Y., Man, H., 2005. Network vulnerability assessment using Bayesian networks, Data mining, intrusion detection, information assurance, and data networks security 2005. *SPIE* 61–71.
- M.S. Lund, B. Solhaug, K. Stølen, M.S. Lund, B. Solhaug, K. Stølen, A guided tour of the CORAS method, *Model-driven risk analysis: the CORAS approach*. (2011) 23–43.
- Mohammadzad, M., Karimpour, J., Mahan, F., 2023. MAGD: minimal attack graph generation dynamically in cyber security. *Comput. Netw.* 236, 110004.
- Nia, M.A., Bahrak, B., Kargahi, M., Fabian, B., 2019. Detecting new generations of threats using attribute-based attack graphs. *IET. Inf. Secur.* 13, 293–303.
- Pape, N., Mansour, C., 2024. PASTA threat modeling for vehicular networks security. In: 2024 7th International conference on information and computer technologies (ICICT). IEEE, pp. 474–478.
- Phillips, C., Swiler, L.P., 1998. A graph-based system for network-vulnerability analysis. In: *Proceedings of the 1998 workshop on New security paradigms*, pp. 71–79.
- Schiele, N.D., Gadyatskaya, O., 2021. arXiv preprint.
- Tippenhauer, N.O., Temple, W.G., Vu, A.H., Chen, B., Nicol, D.M., Kalbarczyk, Z., Sanders, W.H., 2014. Automatic generation of security argument graphs. In: 2014 IEEE 20th Pacific rim international symposium on dependable computing. IEEE, pp. 33–42.
- Widel, W., Hacks, S., Ekstedt, M., Johnson, P., Lagerström, R., 2023. The meta attack language-a formal description. *Comput. Secur.* 130, 103284.
- Zhang, M., Wang, L., Jajodia, S., Singhal, A., Albanese, M., 2016. Network diversity: a security metric for evaluating the resilience of networks against zero-day attacks. *IEEE Transact. Inform. Forens. Sec.* 11, 1071–1086.